

HDOC – Experiment 23

Question

Write a C program that displays the following menu and uses a switch statement to print the basic square/rectangular pattern selected by the user. Use nested loops without arrays. Each pattern may be implemented in a separate parameterless function with no return value.

❖ Understanding problem

We need to write a menu-driven C program that uses:

- switch-case for menu selection.
- Nested loops to generate patterns.
- No arrays.
- A separate parameterless void function for each pattern.
- Validation so that rows and columns must be positive integers.

The four operations are:

1. Solid square
2. Solid rectangle
3. Row-number pattern
4. Column-number pattern
5. Exit

❖ Algorithm

- **Main Algorithm**

1. Start.
2. Declare integer variable choice.
3. Display the menu:
 - 1. Solid square pattern
 - 2. Solid rectangular pattern
 - 3. Row-number pattern
 - 4. Column-number pattern
 - 5. Exit
4. Accept choice from the user.
5. Use a switch statement:
 - If choice = 1, call solidSquare().
 - If choice = 2, call solidRectangle().
 - If choice = 3, call rowNumber().
 - If choice = 4, call columnNumber().
 - If choice = 5, display an exit message.
 - Otherwise, display "Invalid choice."
6. Repeat Steps 3–5 until the user chooses 5.
7. Stop.

- **Algorithm for solidSquare()**

1. Start the function.
2. Input the number of rows n.
3. If $n \leq 0$, display an error message and return.
4. Repeat from $i = 1$ to n:
 - Repeat from $j = 1$ to n:
 - Print *.
 - Move to the next line.
5. Return to main().

- **Algorithm for solidRectangle()**

1. Start the function.
2. Input number of rows m and columns n.
3. If $m \leq 0$ or $n \leq 0$, display an error message and return.
4. Repeat from $i = 1$ to m:
 - Repeat from $j = 1$ to n:
 - Print *.
 - Move to the next line.
5. Return to main().

- **Algorithm for rowNumber()**

1. Start the function.
2. Input number of rows m and columns n.
3. If $m \leq 0$ or $n \leq 0$, display an error message and return.
4. Repeat from $i = 1$ to m:

- Repeat from $j = 1$ to n :
 - Print i .
 - Move to the next line.
5. Return to `main()`.

- **Algorithm for `columnNumber()`**

1. Start the function.
2. Input number of rows m and columns n .
3. If $m \leq 0$ or $n \leq 0$, display an error message and return.
4. Repeat from $i = 1$ to m :
 - Repeat from $j = 1$ to n :
 - Print j .
 - Move to the next line.
5. Return to `main()`.

❖ Test case table

Test Case	Menu Choice	Input(s)	Expected Output	Actual Output	Result
1	1	$n = 4$	4×4 solid square	4×4 solid square	Pass
2	1	$n = 1$	One *	One *	Pass
3	1	$n = 0$	Error message	Error message	Pass
4	2	$m = 3, n = 5$	3×5 solid rectangle	3×5 solid rectangle	Pass
5	2	$m = 1, n = 4$	1×4 solid rectangle	1×4 solid rectangle	Pass
6	2	$m = -2, n = 5$	Error message	Error message	Pass

7	3	m = 4, n = 4	Rows containing 1, 2, 3, 4 respectively	As expected	Pass
8	3	m = 3, n = 2	1 1, 2 2, 3 3	As expected	Pass
9	3	m = 0, n = 4	Error message	Error message	Pass
10	4	m = 4, n = 4	Each row: 1 2 3 4	As expected	Pass
11	4	m = 2, n = 5	Each row: 1 2 3 4 5	As expected	Pass
12	4	m = 3, n = 0	Error message	Error message	Pass
13	6	—	Invalid choice	Invalid choice	Pass
14	5	—	Program terminates	Program terminates	Pass

❖ Program code in C

```
#include <stdio.h>
```

```
/* Function declarations */
```

```
void solidSquare(void);
```

```
void solidRectangle(void);
```

```
void rowNumber(void);
```

```
void columnNumber(void);
```

```
int main(void)
```

```
{
```

```
    int choice;
```

```
    do
```

```
    {
```

```
printf("\n===== PATTERN MENU =====\n");  
printf("1. Solid square pattern\n");  
printf("2. Solid rectangular pattern\n");  
printf("3. Row-number pattern\n");  
printf("4. Column-number pattern\n");  
printf("5. Exit\n");
```

```
printf("Enter your choice: ");  
scanf("%d", &choice);
```

```
switch (choice)  
{  
    case 1:  
        solidSquare();  
        break;  
  
    case 2:  
        solidRectangle();  
        break;  
  
    case 3:  
        rowNumber();  
        break;  
  
    case 4:  
        columnNumber();  
        break;
```

case 5:

printf("Exiting the program.\n");

break;

default:

printf("Invalid choice. Please enter a choice from 1 to 5.\n");

}

} while (choice != 5);

return 0;

}

/* Function to print solid square pattern */

void solidSquare(void)

{

int n, i, j;

printf("Enter the number of rows: ");

scanf("%d", &n);

if (n <= 0)

{

printf("Error: Number of rows must be positive.\n");

return;

}

```

printf("\nSolid Square Pattern:\n");

for (i = 1; i <= n; i++)
{
    for (j = 1; j <= n; j++)
    {
        printf("* ");
    }
    printf("\n");
}

/* Function to print solid rectangular pattern */
void solidRectangle(void)
{
    int m, n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &m);

    printf("Enter the number of columns: ");
    scanf("%d", &n);

    if (m <= 0 || n <= 0)
    {
        printf("Error: Rows and columns must be positive.\n");
        return;
    }
}

```

```
}
```

```
printf("\nSolid Rectangular Pattern:\n");
```

```
for (i = 1; i <= m; i++)
```

```
{
```

```
    for (j = 1; j <= n; j++)
```

```
    {
```

```
        printf("* ");
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
}
```

```
/* Function to print row-number pattern */
```

```
void rowNumber(void)
```

```
{
```

```
    int m, n, i, j;
```

```
    printf("Enter the number of rows: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter the number of columns: ");
```

```
    scanf("%d", &n);
```

```
    if (m <= 0 || n <= 0)
```

```
{
```

```
    printf("Error: Rows and columns must be positive.\n");
    return;
}

printf("\nRow-Number Pattern:\n");

for (i = 1; i <= m; i++)
{
    for (j = 1; j <= n; j++)
    {
        printf("%d ", i);
    }
    printf("\n");
}
}
```

```
/* Function to print column-number pattern */
void columnNumber(void)
{
    int m, n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &m);

    printf("Enter the number of columns: ");
    scanf("%d", &n);
```

```
if (m <= 0 || n <= 0)
{
    printf("Error: Rows and columns must be positive.\n");
    return;
}

printf("\nColumn-Number Pattern:\n");

for (i = 1; i <= m; i++)
{
    for (j = 1; j <= n; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}
}
```

❖ Compile, Execute and Evaluate

```
(kali㉿kali)-[~/Desktop/exp23]
$ vi pattern.c

(kali㉿kali)-[~/Desktop/exp23]
$ gcc pattern.c -o pattern

(kali㉿kali)-[~/Desktop/exp23]
$ ./pattern
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 1
Enter the number of rows: 4
```

Solid Square Pattern:

```
* * * *
* * * *
* * * *
* * * *
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 1
Enter the number of rows: 1
```

Solid Square Pattern:

```
*
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 1
Enter the number of rows: 0
Error: Number of rows must be positive.
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 2
Enter the number of rows: 3
Enter the number of columns: 5
```

Solid Rectangular Pattern:

```
* * * * *
* * * * *
* * * * *
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 2
Enter the number of rows: 1
Enter the number of columns: 4
```

Solid Rectangular Pattern:

```
* * * *
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 2
Enter the number of rows: -2
Enter the number of columns: 5
Error: Rows and columns must be positive.
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 3
Enter the number of rows: 4
Enter the number of columns: 4
```

Row-Number Pattern:

```
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 3
Enter the number of rows: 3
Enter the number of columns: 2
```

Row-Number Pattern:

```
1 1
2 2
3 3
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 3
Enter the number of rows: 0
Enter the number of columns: 4
Error: Rows and columns must be positive.
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 4
Enter the number of rows: 4
Enter the number of columns: 4
```

Column-Number Pattern:

```
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
```

```
===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 4
```

```
Enter the number of rows: 2
Enter the number of columns: 5

Column-Number Pattern:
1 2 3 4 5
1 2 3 4 5

===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 4
Enter the number of rows: 3
Enter the number of columns: 0
Error: Rows and columns must be positive.

===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 6
Invalid choice. Please enter a choice from 1 to 5.

===== PATTERN MENU =====
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 5
Exiting the program.

(kali㉿kali)-[~/Desktop/exp23]
$
```